Chapter 4

Higher Inductive Types

Coquand et al. (2018) は、cubical type theory を高次帰納型（*higher inductive types*）へ拡張する方法を与えた。その鍵が、まさに Section 3.3–Section 3.4 で採った「合成を hcomp と transp に分ける」提示である：高次帰納型では、合成構造を「構成子としての hcomp」と「構成子ごとに与える transp の κ-簡約」として指定できる。

4.1 A Common Pattern for Higher Inductive Types

CHM の考える高次帰納型の例はすべて共通のパターンに従う。これは cubical type theory における高次帰納型の構文的スキーマの定式化への第一歩とみなせるが、スキーマの正確な定式化とその意味論的対応物は future work とされている。

各高次帰納型 $D(\vec{z} : \vec{P})$ は、（ambient context Γ 上の）パラメータの telescope $\vec{z} : \vec{P}$ と構成子のリスト $\vec{c}$ で指定される[*1]。各構成子 c は次のデータで指定される：

$$c \,:\, (\vec{x} : \vec{A}(\vec{z}))\ [\vec{i}]\ D(\vec{z}) \qquad \text{境界データ } \varphi(\vec{i}) \mapsto e(\vec{z}, \vec{x}, \vec{i})$$

ここで telescope $\vec{x} : \vec{A}$ は c の引数の型を指定する（propositional truncation のような再帰的高次帰納型では、D 自身が $\vec{A}$ に現れてよい）。名前のリスト $\vec{i}$ の長さは、c が導入する立方体の次元を指定する：長さが 0, 1, 2 のとき、c をそれぞれ point 構成子、path 構成子、square 構成子と呼ぶ。境界データ $\varphi \mapsto e$ は構成子 c の端点を指定する：φ は自由変数が $\vec{i}$ に含まれる $\mathbb{F}$ の元であり、e は部分元

$$\vec{z} : \vec{P},\ \vec{x} : \vec{A}(\vec{z}),\ \vec{i} : \mathbb{I},\ \varphi(\vec{i}) \vdash e(\vec{z}, \vec{x}, \vec{i}) : D(\vec{z})$$

であって、リスト $\vec{c}$ の中で先行する構成子（および後述の hcomp）のみに言及するものである。

telescope $\vec{z} : \vec{P}$ の各インスタンス $\vec{u} : \vec{P}$ に対して $D(\vec{u})$ は型であり、上のように指定された構成子 c の導入規則は（ambient context を省略して）次で与えられる：

$$\frac{\vec{v} : \vec{A}(\vec{u}) \quad \vec{r} : \mathbb{I}}{c\,\vec{v}\,\vec{r} : D(\vec{u})}$$

さらに $\varphi(\vec{r}) = 1_{\mathbb{F}}$ が成り立つ場合には、判断的等値性

$$c\,\vec{v}\,\vec{r} =_{\partial} e(\vec{u}, \vec{v}, \vec{r}) \;:\; D(\vec{u})$$

[*1] 文脈 Γ 上の *telescope* $x_1 : A_1, \ldots, x_n : A_n$（$\vec{x} : \vec{A}$ と書く）とは、$\Gamma, \vec{x} : \vec{A}$ が well-formed な文脈となるような（空でもよい）対象変数宣言のリストである。したがって $\vec{x} : \vec{A}$ は文脈制限 Δ, φ も名前宣言 $i : \mathbb{I}$ も含まない。

が成り立つ（Section 3.6 の ∂-等値性にあたる）。この判断的等値性のため、関数 $f : (x : D(\vec{u})) \to P(x)$ を定義するときには、f がこの等値性を保存すること、すなわち

$$\varphi(\vec{r}) \vdash f(c\,\vec{v}\,\vec{r}) = f(e(\vec{u},\vec{v},\vec{r})) : P(c\,\vec{v}\,\vec{r})$$

が保証されなければならない。この要件は $D(\vec{u})$ の除去規則（eliminator）の型付け規則で扱われるべきものであるが、その一般的定式化は（「extension types」に類する拡張を要するため）future work とされている。

Remark 37 の見取り図で予告した第三の場合が、まさに高次帰納型である：path 構成子があるため、側面は構成子頭とは限らず、hcomp の κ-簡約は与えられない。そこで homogeneous composition の構造は「構成子として」導入される。といっても、新しい規則は要らない。$A := D(\vec{u})$ に対する hcomp の型付けと ∂-等値性は、Definition 30 がそのまま与える（$D(\vec{u})$ は Γ で型付けされ、方向 i に依存しない—homogeneous composition が適用できるのはこのためである。i は側面 v の中には自由に現れてよい）。高次帰納型に固有なのは、次の 2 点だけである。第一に、この hcomp には κ-簡約を**与えない**。したがって $\mathtt{hcomp}^i_{D(\vec{u})}\,[\varphi \mapsto v]\,v_0$ という形の元それ自体が $D(\vec{u})$ の標準形となる（暗黙の構成子とみなしてよい）。第二に、その代償として、$D(\vec{u})$ からの除去写像には hcomp との可換性を述べる計算規則が付く：$f : D(\vec{u}) \to C$ に対し $f(\mathtt{hcomp}^i_{D(\vec{u})}\,[\varphi \mapsto v]\,v_0) \to_\beta \mathtt{hcomp}^i_C\,[\varphi \mapsto f(v)]\,(f(v_0))$ の形の規則である（Section 4.5 の $\mathbb{Z}$ の除去規則を参照。依存除去の一般形は上述のとおり future work に属する）。以下の例では、この hcomp 構成子を毎回書かないが、考える各高次帰納型の定義にはつねに含まれているものとする。

Remark 37 の見取り図の第二の場合で触れた「hcomp を残す設計」とは、自然数型のような通常の帰納型にこの扱いを適用することにあたる：hcomp を再帰で消去する代わりに、構成子 $\mathtt{hcomp}^i_{\mathbb{N}}$ を持たせてもよい。この「弱い」形の自然数型は通常のものと同値であり、したがって（univalence により）等しいことが証明できる。

移送については、Section 3.4 の transp がそのまま用いられる：$\Gamma, i : \mathbb{I} \vdash \vec{u} : \vec{P}$ が与えられたとき、線 $A := D(\vec{u})$ に対する $\mathtt{transp}^i A\,\varphi\,u_0$ である。ただしその κ-簡約は、Section 3.4 のような型の形ごとの再帰では与えられず、u_0 の取りうる形ごとに与えられる：すなわち構成子 c ごとに 1 つ、そして hcomp 構成子に対して 1 つである。後者は

$$\begin{aligned}&\mathtt{transp}^i A\,\varphi\,(\mathtt{hcomp}^j_{A[0/i]}\,[\psi \mapsto u]\,u_0)\\ &\quad\to_\kappa \mathtt{hcomp}^j_{A[1/i]}\,[\psi \mapsto \mathtt{transp}^i A\,\varphi\,u]\,(\mathtt{transp}^i A\,\varphi\,u_0)\end{aligned}$$

で与えられる（i と j はともに束縛変数なので、名前替えにより $i \neq j$ と仮定してよい）。hcomp の場合はすべての例で同じ形なので、以下では各高次帰納型の transp の定義からは省略する。

Section 3.5 の comp の定義は、hcomp と transp（と squeeze）のみを用いていたことを思い出そう。したがって、高次帰納型の合成は、「構成子に適用された場合の transp」を与えるだけで定まる。

4.2 The Circle

共通パターンの最初の例として、最も単純な高次帰納型である円周（circle）S^1 を見よう。

Definition 50 (The Circle S^1)**.** 高次帰納型 S^1 は、パラメータなしで、次の 2 つの構成子により定義される：

$$\frac{\Gamma \ \mathit{context}}{\Gamma \vdash \mathtt{base} : S^1} \qquad \frac{\Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \mathtt{loop}\, r : S^1}$$

ただし `loop` は次の境界の等値性判断（∂-等値性）をともなう：

$$\Gamma \vdash \mathtt{loop}\, 0 =_\partial \mathtt{base} : S^1 \qquad \Gamma \vdash \mathtt{loop}\, 1 =_\partial \mathtt{base} : S^1$$

（共通パターンに従い、構成子 $\mathtt{hcomp}^i_{S^1}$ も定義に含まれる。）

`base` は point 構成子、`loop` は path 構成子であり、$\langle i \rangle\, \mathtt{loop}\, i : \mathrm{Path}\, S^1\, \mathtt{base}\, \mathtt{base}$ を与える。すなわち S^1 は、1 つの点と、その点から自分自身への 1 本の path とからなる型である。名前のとおり、これは円周を表す。

ここで、等号型（Section 3.7）との違いが鮮明になる。等号型のもとで UIP を仮定すれば、`base` から `base` への証明はすべて $\mathtt{refl}(\mathtt{base})$ に等しいことになり、`loop` は $1_{\mathtt{base}}$ と区別できない。しかし Path 型のもとでは両者は区別され、実際 $\mathrm{Path}\,(\mathrm{Path}\, S^1\, \mathtt{base}\, \mathtt{base})\, 1_{\mathtt{base}}\, (\langle i \rangle\, \mathtt{loop}\, i)$ は inhabited でないことが証明できる[*2]。これが Section 3.7 で述べた「型は集合ではなく高次の構造をもちうる」ことの最も簡単な実例である。

S^1 はパラメータをもたないから、線が定数の場合しか生じず、transp の κ-簡約は自明である：

$$\mathtt{transp}^i\, S^1\, \varphi\, u_0 \to_\kappa u_0$$

したがって S^1 上の合成もすべて homogeneous であり、`hcomp` だけで足りる。

型 C への関数 $f : S^1 \to C$ を定義するには、点 $c : C$ とループ $l : \mathrm{Path}\, C\, c\, c$ を与えればよく、$f(\mathtt{base}) \to_\beta c$、$f(\mathtt{loop}\, r) \to_\beta l\, r$、および hcomp との可換性が成り立つ。依存版（motive $P : S^1 \to \mathrm{type}$）では、`loop` に対する証明義務は P 自身が `loop` に沿って動くため、Section 2.3 の依存 path

$$\bar{l} : \mathrm{PathP}^i\, P(\mathtt{loop}\, i)\, \bar{c}\, \bar{c} \qquad (\bar{c} : P(\mathtt{base}) \text{ に対して})$$

となる。これが PathP を導入した理由である。

Exercise 51. $\ell \overset{def}{\equiv} \langle i \rangle\, \mathtt{loop}\, i$ とおく。本文の recursion principle のデータとして $c \overset{def}{\equiv} \mathtt{base}$、$l \overset{def}{\equiv} \ell \cdot \ell$（Section 3.3 の合成）を与えると、「2 周する」写像 $\mathrm{double} : S^1 \to S^1$ が定義される。(1) $\ell \cdot \ell : \mathrm{Path}\, S^1\, \mathtt{base}\, \mathtt{base}$ が導出できること—合成の hcomp の前提（Lemma 31）が満たされること—を確かめよ。(2) $\mathrm{double}(\mathtt{loop}\, r)$ が β-規則と (Pathβ) でどんな項に簡約されるかを書き下せ。それは構成子頭ではなく hcomp 頭の項である。r に 0, 1 を代入し、∂-等値性によりその値が `base` に等しいことを検算せよ。(3) 同様に、$l \overset{def}{\equiv} \ell^{-1} = \langle i \rangle\, \mathtt{loop}\, (1\text{-}i)$（Exercise 16）をデータとして「逆回り」の写像を定義し、こちらは $\mathtt{loop}\, r$ の行き先が構成子頭のままであることを確かめよ。

[*2] S^1 の基本群が $\mathbb{Z}$ であること（$\pi_1(S^1) = \mathbb{Z}$）の帰結である。homotopy type theory における証明は The Univalent Foundations Program (2013) の第 8 章を、cubical type theory での構成的な扱いは Mörtberg (2021) を参照。本書では立ち入らない。

4.3 The Torus

Section 4.1 のスキーマは、長さ 2 の名前リストをもつ構成子—square 構成子—を許すが、ここまでの例はいずれも長さ 1（path 構成子）であった。square 構成子の代表的な実例がトーラス T^2 である。

Definition 52 (The Torus T^2)**.** 高次帰納型 T^2 は、パラメータなしで、次の構成子により定義される（$k = 1, 2$）：

$$\frac{\Gamma \ \mathit{context}}{\Gamma \vdash \mathtt{point} : T^2} \qquad \frac{\Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \mathtt{line}_k\, r : T^2} \qquad \frac{\Gamma \vdash r : \mathbb{I} \quad \Gamma \vdash s : \mathbb{I}}{\Gamma \vdash \mathtt{square}\, r\, s : T^2}$$

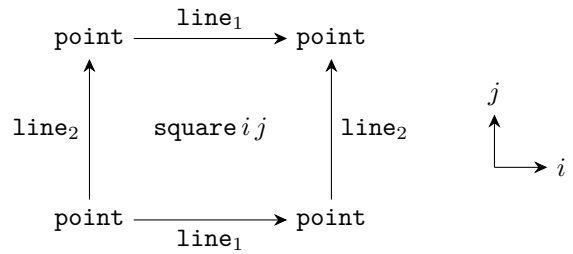
ただし $\mathtt{line}_k$ と square は次の境界の等値性判断（∂-等値性）をともなう：

$$\Gamma \vdash \mathtt{line}_k\, 0 =_\partial \mathtt{point} : T^2 \qquad \Gamma \vdash \mathtt{line}_k\, 1 =_\partial \mathtt{point} : T^2$$

$$\Gamma \vdash \mathtt{square}\, 0\, s =_\partial \mathtt{line}_2\, s : T^2 \quad \Gamma \vdash \mathtt{square}\, 1\, s =_\partial \mathtt{line}_2\, s : T^2$$
$$\Gamma \vdash \mathtt{square}\, r\, 0 =_\partial \mathtt{line}_1\, r : T^2 \quad \Gamma \vdash \mathtt{square}\, r\, 1 =_\partial \mathtt{line}_1\, r : T^2$$

（共通パターンに従い、構成子 $\mathtt{hcomp}^i_{T^2}$ も定義に含まれる。）

square の境界の 4 本の等値性は、対辺を貼り合わせた正方形というトーラスの標準的な描像をそのまま読み上げたものである：下辺（$j = 0$）と上辺（$j = 1$）がともに $\mathtt{line}_1$、左辺（$i = 0$）と右辺（$i = 1$）がともに $\mathtt{line}_2$ である。



同じ型を等号型の語彙で与えると、次のようになる（The Univalent Foundations Program (2013) §6.6）：

$$\mathtt{point} : T^2 \qquad p, q : \mathtt{point} =_{T^2} \mathtt{point} \qquad \mathtt{surf} : p \bullet q = q \bullet p$$

ここで注目すべきは、最後の構成子を述べるために path の合成 $\bullet$（本書の $p \cdot q$ にあたる、J から導出される演算）が必要になることである。型の定義が導出済みの演算に依存し、しかも $p \bullet q$ という書き方には合成の結合の向きの選択が入り、2 本の辺は非対称に（一方は等式の左辺、他方は右辺に）現れる。cubical のスキーマでは、square は単に区間変数を 2 つもつ項であり、その境界を 4 本の ∂-等値性で対等に読み上げるだけでよい—導出された演算も J も現れない。n 次元の構成子は n 個の区間変数をもつ項である、という形で、次元が上がっても枠組みは変わらないのである。

この記法上の差は、証明の労力の差として現れる。たとえば $T^2 \simeq S^1 \times S^1$ は、等号型の語彙では独立の論文を要する仕事だったが（Sojakova による 2016 年の ACM TOCL 論文）、cubical type theory では大幅に短い証明が知られている。なお依存除去

では、square の像の型が 2 つの区間変数の両方に依存するため、証明義務は依存 path (Section 2.3) の 2 次元版—PathP の入れ子—となる。また T^2 はパラメータをもたないから、S^1 と同様、transp の κ-簡約は自明であり、T^2 上の合成はすべて homogeneous である。

Exercise 53. S^1 の場合 (Section 4.2) にならって、型 C への関数 $g : T^2 \to C$ を定義するために与えるべきデータを構成子ごとに書き出せ。square に対して要求されるのは、区間変数を 2 つもち、Definition 52 の 4 本の境界とデータの行き先が整合するような項である。そのうえで、第 1 の射影 $\mathrm{pr}_1 : T^2 \to S^1$ を、データ

$$\mathtt{point} \mapsto \mathtt{base} \qquad \mathtt{line}_1\, r \mapsto \mathtt{loop}\, r \qquad \mathtt{line}_2\, s \mapsto \mathtt{base} \qquad \mathtt{square}\, r\, s \mapsto \mathtt{loop}\, r$$

で定義し、square に要求される 4 本の境界条件が、loop の ∂-等値性によりすべて満たされることを検算せよ。また、$\mathtt{line}_2$ を loop へ、$\mathtt{line}_1$ を定値へ送る第 2 の射影 pr_2 のデータも与えよ。(幾何学的には、トーラスの 2 つの周回方向への射影である。対にした $T^2 \to S^1 \times S^1$ が同値になることが、先に触れた Sojakova の定理である。)

4.4 Propositional Truncation

S^1 と T^2 は、path 構成子によって型に**新しい** path を追加する例であった。同じ仕組みは、逆向きの目的—既存の元どうしを path で結んで**区別を消す**こと—にも使える。その最小の例が *propositional truncation* (命題切り詰め) $\|A\|$ である。

まず語彙を整える。型 A が命題 (proposition) であるとは、どの 2 元も path で結ばれることをいい、集合 (set) であるとは、どの path 型も命題であることをいう：

$$\mathbf{isProp}(A) \stackrel{def}{\equiv} (x : A) \to (y : A) \to \mathrm{Path}\, A\, x\, y$$
$$\mathbf{isSet}(A) \stackrel{def}{\equiv} (x : A) \to (y : A) \to \mathbf{isProp}(\mathrm{Path}\, A\, x\, y)$$

(この階層の一段下にある「元が本質的に 1 つ」を表す **isContr** は、Chapter 5 で同値の定義とともに導入する。) Section 4.2 の言葉でいえば、UIP が成り立つ型が集合であり、S^1 は集合でない型の実例であった。これまで非形式的に用いてきた「集合」は、以後この **isSet** を正式の定義とする。なお、ここでの「命題」は型の h-level についての術語であり、「命題的 (propositional) 等値性」—判断的等値性との対比で、「規則ではなく項として与えられる等値性」を指す言い回し—の「命題」とは別の用法であることに注意してほしい。

Definition 54 (Propositional Truncation)**.** 型 $\Gamma \vdash A : \mathbf{type}$ に対し、高次帰納型 $\|A\|$ は次の 2 つの構成子により定義される：

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash |a| : \|A\|} \qquad \frac{\Gamma \vdash x : \|A\| \quad \Gamma \vdash y : \|A\| \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \mathtt{squash}\, x\, y\, r : \|A\|}$$

ただし squash は次の境界の等値性判断 (∂-等値性) をともなう：

$$\Gamma \vdash \mathtt{squash}\, x\, y\, 0 =_\partial x : \|A\| \qquad \Gamma \vdash \mathtt{squash}\, x\, y\, 1 =_\partial y : \|A\|$$

(共通パターンに従い、構成子 $\mathtt{hcomp}^i_{\|A\|}$ も定義に含まれる。)

S^1・T^2 と違い、squash は**定義中の型** $\|A\|$ **自身の元** $x,\ y$ を引数にとる。これは、Section 4.1 のスキーマが telescope $\vec{x} : \vec{A}$ に D 自身の現れることを許していた、**再帰的**高次帰納型の最初の実例である。squash の与える path 抽象は

$$\langle i \rangle\, \mathtt{squash}\, x\, y\, i\ :\ \mathrm{Path}\, \|A\|\, x\, y$$

だから、$\lambda x.\lambda y.\langle i \rangle\, \mathtt{squash}\, x\, y\, i$ がそのまま $\mathbf{isProp}(\|A\|)$ の証明である。すなわち $\|A\|$ は、構成子によって**命題にされた**型であり、「A に元があるか」だけを覚えて、どの元か・どの証明かの区別を残さない。

型 C への関数 $\tilde{f} : \|A\| \to C$ を定義するには、$f : A \to C$ と、**行き先が命題であることの証明** $h : \mathbf{isProp}(C)$ を与えればよく、

$$\tilde{f}(|a|) \to_\beta f\, a \qquad \tilde{f}(\mathtt{squash}\, x\, y\, r) \to_\beta h\, (\tilde{f}\, x)\, (\tilde{f}\, y)\, r$$

および hcomp との可換性が成り立つ。第 2 の規則の右辺が正しい境界をもつことは、h の与える path の端点規則から従う：$r = 0$ で $h\, (\tilde{f}\, x)\, (\tilde{f}\, y)\, 0 =_\partial \tilde{f}\, x$ となり、構成子の境界 $\mathtt{squash}\, x\, y\, 0 =_\partial x$ の像と一致する（$r = 1$ も同様）。S^1 の recursion principle ではループの行き先 l を自由に選べたのに対し、ここでは squash の行き先が h によって決まってしまう—「除去の際に**行き先の型へ条件を課す**」構成子の最初の例である。依存版（motive $P : \|A\| \to \mathrm{type}$ で各 $P(w)$ が命題）では、squash に対する証明義務は Section 2.3 の依存 path となり、命題性と transp からつねに埋められる（Exercise 56）。

この定義の見返りは計算的である：$\tilde{f}(|a|) = f\, a$ が**判断的**に成り立つので、切り詰めを通した証明からも計算が走り、canonicity が保たれる。たとえば選言や存在を $P \vee Q \overset{def}{\equiv} \|P + Q\|$ の形で定義したとき、その証明項は実際に分岐先を計算する。

最後に、次節で繰り返し使う補題を証明しておく。命題性は、1 段上の h-level を道連れにする：

Lemma 55. $\mathbf{isProp}(A)$ ならば $\mathbf{isSet}(A)$ である。

Proof. $h : \mathbf{isProp}(A)$、$x, y : A$、$p, q : \mathrm{Path}\, A\, x\, y$ とする。求める $\mathrm{Path}\, (\mathrm{Path}\, A\, x\, y)\, p\, q$ を、$j : \mathbb{I}, i : \mathbb{I}$ のもとで定義される、4 つの側面をもつ開いた箱の蓋

$$\begin{aligned}\mathtt{hcomp}^k_A\, [\, &(i{=}0) \mapsto h\, x\, x\, k,\ (i{=}1) \mapsto h\, x\, y\, k,\\ &(j{=}0) \mapsto h\, x\, (p\, i)\, k,\ (j{=}1) \mapsto h\, x\, (q\, i)\, k\,]\ x\end{aligned}$$

を $\langle j \rangle\, \langle i \rangle \cdots$ と抽象して与える。Lemma 31 の前提の検算：底面は定値 x であり、各側面の始端は端点規則により $h\, x\, z\, 0 =_\partial x$ の形だから、底面と接続する。側面どうしの重なりについては、たとえば $(i{=}0) \wedge (j{=}0)$ の上で $p\, 0 =_\partial x$ により $h\, x\, (p\, 0) = h\, x\, x$ が判断的に成り立ち、両立する（残り 3 箇所も同様。$(i{=}0) \wedge (i{=}1)$ などは $0_{\mathbb{F}}$ なので自明）。蓋の 4 辺は、各側面の終端 $h\, x\, z\, 1 =_\partial z$ により、$i{=}0$ で x、$i{=}1$ で y、$j{=}0$ で $p\, i$、$j{=}1$ で $q\, i$ となる。これは「p から q への、端点を固定した path」の型が要求する境界そのものである。 □

Exercise 56. （1）$g : A \to B$ に対し、functoriality $\|g\| : \|A\| \to \|B\|$ を recursion principle により定義せよ（ヒント：行き先 $\|B\|$ の命題性は squash 自身が与える）。（2）依存版の除去において squash に課される証明義務

$$\mathrm{PathP}^i\, P(\mathtt{squash}\, x\, y\, i)\, (\mathrm{elim}\, x)\, (\mathrm{elim}\, y)$$

が、各 $P(w)$ の命題性から埋められることを示せ（ヒント：Exercise 48 の対応で移送を経由した通常の path に直し、$P(\mathtt{squash}\, x\, y\, 1)$ の命題性を用いる）。

4.5 The Integers

整数は自然数の対によって「差」として表現できる。差分（difference）とは自然数の組 $\langle m, n\rangle$ のことであり、差分の間には同値関係

$$\langle m, n\rangle \sim_d \langle p, q\rangle \quad \stackrel{def}{\equiv} \quad m + q = p + n$$

が定まる（$\langle 7, 2\rangle \sim_d \langle 15, 10\rangle$ は $7 + 10 = 15 + 2$ による）。古典的な構成では、整数とはこの同値関係による**商**、すなわち差分の同値類 $[\langle m, n\rangle]$ のことであった。setoid によるアプローチはこの商をとらず、同値関係を構造として持ち回る。高次帰納型を用いれば、第 3 の道が開ける：**同値関係そのものを path 構成子として型の定義に組み込む**のである。「同値類をとる」操作が point 構成子 $[\langle m, n\rangle]$ に、「$\sim_d$ ならば同じ類」という商の性質が path 構成子 `eq` に、そして「商集合は集合である」ことが Section 4.4 の squash の 2 次元版である構成子 `trunc` に、それぞれ対応する。

Definition 57 (The Integer Type $\mathbb{Z}$)**.** 高次帰納型 $\mathbb{Z}$ は、パラメータなしで、次の 3 つの構成子により定義される：

$$\frac{\Gamma \vdash m : \mathbb{N} \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash [\langle m, n\rangle] : \mathbb{Z}} \qquad \frac{\Gamma \vdash m, n, p, q : \mathbb{N} \quad \Gamma \vdash e : \mathrm{Path}\,\mathbb{N}\,(m+q)\,(p+n) \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \mathtt{eq}\,m\,n\,p\,q\,e\,r : \mathbb{Z}}$$

$$\frac{\Gamma \vdash x, y : \mathbb{Z} \quad \Gamma \vdash p, q : \mathrm{Path}\,\mathbb{Z}\,x\,y \quad \Gamma \vdash r, s : \mathbb{I}}{\Gamma \vdash \mathtt{trunc}\,x\,y\,p\,q\,r\,s : \mathbb{Z}}$$

ただし `eq` と `trunc` は次の境界の等値性判断（∂-等値性）をともなう：

$$\mathtt{eq}\,m\,n\,p\,q\,e\,0 =_\partial [\langle m, n\rangle] \qquad \mathtt{eq}\,m\,n\,p\,q\,e\,1 =_\partial [\langle p, q\rangle]$$

$$\mathtt{trunc}\,x\,y\,p\,q\,0\,s =_\partial p\,s \quad \mathtt{trunc}\,x\,y\,p\,q\,1\,s =_\partial q\,s$$
$$\mathtt{trunc}\,x\,y\,p\,q\,r\,0 =_\partial x \quad \mathtt{trunc}\,x\,y\,p\,q\,r\,1 =_\partial y$$

（共通パターンに従い、構成子 $\mathtt{hcomp}^i_{\mathbb{Z}}$ も定義に含まれる。）

3 つの構成子を順に読もう。$[\langle m, n\rangle]$ は差分 $\langle m, n\rangle$ の類であり、整数 $m - n$ を表す。`eq` は、**同値関係 $\sim_d$ の定義をそのまま前提に写した** path 構成子である：$\sim_d$ の witness e を与えるごとに

$$\langle i\rangle\, \mathtt{eq}\,m\,n\,p\,q\,e\,i \;:\; \mathrm{Path}\,\mathbb{Z}\,[\langle m, n\rangle]\,[\langle p, q\rangle]$$

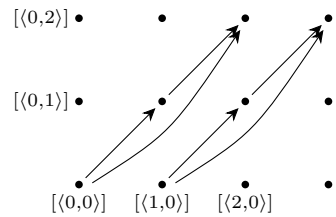
が得られる。古典的構成で「$x \sim_d y$ ならば $[x] = [y]$」として証明抜きで使っていた商の基本性質が、ここでは構成子である。`trunc` は Section 4.4 の `squash` の 2 次元版であり、$\mathbb{Z}$ の任意の 2 本の平行な path を正方形で結ぶ。すなわち

$$\lambda x.\lambda y.\lambda p.\lambda q.\langle r\rangle\,\langle s\rangle\,\mathtt{trunc}\,x\,y\,p\,q\,r\,s \;:\; \mathbf{isSet}(\mathbb{Z})$$

であり、$\mathbb{Z}$ は構成子によって**集合にされた**型である。「商集合」の「集合」の部分を担うのが `trunc` だ、と読める。なお `eq` は、引数の型に Path 型（witness e）が現れる**引数つきの** path 構成子であり、`trunc` は定義中の $\mathbb{Z}$ 自身の元と path を引数にとる**再帰的**

構成子である（Section 4.4 参照）[*3]。

幾何学的には、$\mathbb{Z}$ は格子 $\mathbb{N}\times\mathbb{N}$ の各点に point $[\langle m,n\rangle]$ を置き、同じ整数を表す差分の**どの 2 つの間にも** `eq` の辺を張り、さらに平行な path の間を `trunc` の正方形で埋めたものである（図には整数 0 と 1 の成分の辺の一部だけを示す。横方向が m、縦方向が n）：



各連結成分が 1 つの整数に対応する。隣どうしの辺とそれを飛び越す辺が閉路をなしていることが見て取れる—これが `trunc` を要する理由であった（前掲の脚注）。

また、$\mathbb{Z}$ はパラメータをもたないから、線が定数の場合しか生じず、transp の κ-簡約（Section 4.1）は自明である：$\mathtt{transp}^i\,\mathbb{Z}\,\varphi\,u_0 \to_\kappa u_0$。以下の構成で用いる合成はすべて homogeneous であり、hcomp だけで足りる。

trunc からは、後で繰り返し使う一般的な帰結を先に引き出しておく。

Lemma 58 (Squares in a Set)**.** 型 A が集合であるとき、A の中の任意の正方形の境界—すなわち 4 つの path であって、隣り合うものの端点が一致しているもの—は充填できる。

確認：正方形の境界が与えられたとき、そのうち 3 辺を hcomp で合成して 4 辺目と同じ端点をもつ path を作り、集合性によりこの 2 つの path を結ぶ path（すなわち正方形）を得て、これと合成の充填（hfill）とを貼り合わせればよい。

Corollary 59. $\mathbb{Z}$ の中の任意の正方形の境界は充填できる。（`trunc` の与える $\mathbf{isSet}(\mathbb{Z})$ と Lemma 58 による。）

■除去 `trunc` を構成子にもつ型からの除去には、Section 4.4 で見た「行き先への条件」が 2 次元版で現れる。型 C への関数 $f:\mathbb{Z}\to C$ を定義するには、C が**集合であること**の証明 $h_C:\mathbf{isSet}(C)$ と、次の 2 つのデータを与えればよい：

$$d : (m:\mathbb{N})\to(n:\mathbb{N})\to C$$
$$d_{\mathrm{eq}} : (m,n,p,q:\mathbb{N})\to (\mathrm{Path}\,\mathbb{N}\,(m+q)\,(p+n))\to \mathrm{Path}\,C\,(d\,m\,n)\,(d\,p\,q)$$

このとき f は次の簡約規則を満たす：

$$f([\langle m,n\rangle])\to_\beta d\,m\,n$$
$$f(\mathtt{eq}\,m\,n\,p\,q\,e\,r)\to_\beta d_{\mathrm{eq}}\,m\,n\,p\,q\,e\,r$$
$$f(\mathtt{trunc}\,x\,y\,p\,q\,r\,s)\to_\beta h_C\,(fx)\,(fy)\,(\langle k\rangle\,f(p\,k))\,(\langle k\rangle\,f(q\,k))\,r\,s$$

[*3] `trunc` が省けないことは次のようにわかる。`eq` は同値関係の**すべての** witness に対して辺を張るから、たとえば $[\langle 0,0\rangle]$ と $[\langle 2,2\rangle]$ は、直接の辺でも、$[\langle 1,1\rangle]$ 経由の合成でも結ばれ、閉路が生じる。truncation なしではこの閉路が S^1 の loop のような非自明な path として残り、$\mathbb{Z}$ は集合にならない。生成する辺を各 (a,b) に 1 本ずつに絞れば閉路は消え、truncation なしで集合になる—それが Section 5.1 で与える別定義（cancel 版）である。

（および hcomp との可換性。境界の整合は、d_{eq} と h_C の与える path・正方形の端点規則から、Section 4.4 の場合と同様に確かめられる。）d_{eq} が要求するものは、古典的構成の **well-definedness**—「関数の値が代表元の取り方に依らない」—の証明義務そのものである。setoid 構成で関数ごとに証明していた両立性補題が、ここでは recursion principle の入力データとして現れる。

次の補題の準備として、依存 path と h-level の関係を 2 つ見ておく。

Lemma 60 (Paths over a Line of Propositions)**.** 型の線 $\Gamma, i : \mathbb{I} \vdash B$ が各点で命題であるとき、任意の端点 $a : B[0/i]$, $b : B[1/i]$ に対して $\mathrm{PathP}^i\, B\, a\, b$ の元が構成できる。

確認：Exercise 56 (2) は線 $P(\mathtt{squash}\, x\, y\, i)$ についての問題であるが、その構成が用いるのは Exercise 48 の対応と線の終点の命題性だけであり、一般の線 B でもそのまま通る。

Lemma 61 (Paths over a Line of Sets)**.** 型の線 $\Gamma, j : \mathbb{I} \vdash A$ について $A[1/j]$ が集合であるとき、端点を固定した $\mathrm{PathP}^j\, A\, u_0\, u_1$ は命題である。

確認：Section 2.3 の同値により、$\mathrm{PathP}^j\, A\, u_0\, u_1$ は $\mathrm{Path}\, A[1/j]\, (\mathtt{transp}^j A\, 0_{\mathbb{F}}\, u_0)\, u_1$ と同値であり、後者は $A[1/j]$ の集合性により命題である。命題と同値な型は命題である。

示したいものが**命題**のときには、除去の義務はさらに軽くなる：

Lemma 62 (Elimination into Propositions)**.** $P : \mathbb{Z} \to \mathrm{type}$ が各点で命題（すべての z で $\mathbf{isProp}(P(z))$）であるとき、$(z : \mathbb{Z}) \to P(z)$ を構成するには、point 構成子の場合

$$g : (m : \mathbb{N}) \to (n : \mathbb{N}) \to P([\langle m, n \rangle])$$

だけを与えればよい。

証明の概略：$f : (z : \mathbb{Z}) \to P(z)$ は、構成子ごとに値を与える依存版の除去で定義する：

$$\begin{aligned} &f([\langle m, n \rangle]) \to_\beta g\, m\, n \\ &f(\mathtt{eq}\, m\, n\, p\, q\, e\, r) \to_\beta d_{\mathrm{eq}}\, m\, n\, p\, q\, e\, r \\ &f(\mathtt{trunc}\, x\, y\, p\, q\, r\, s) \to_\beta d_{\mathrm{trunc}}\, x\, y\, p\, q\, r\, s \end{aligned}$$

ここで d_{eq}・d_{trunc} が満たすべき型（証明義務）は

$$\begin{aligned} &\mathrm{PathP}^i\, P(\mathtt{eq}\, m\, n\, p\, q\, e\, i)\, (g\, m\, n)\, (g\, p\, q) \\ &\mathrm{PathP}^r\, \big(\mathrm{PathP}^s\, P(\mathtt{trunc}\, x\, y\, p\, q\, r\, s)\, (f\, x)\, (f\, y)\big)\, (\langle s \rangle\, f(p\, s))\, (\langle s \rangle\, f(q\, s)) \end{aligned}$$

である。内側の PathP が意味をもつのは、trunc の境界により $s = 0$ で $P(x)$、$s = 1$ で $P(y)$ となるからである。また hcomp については、motive が動くので hcomp ではなく comp が現れる：

$$f(\mathtt{hcomp}^i_{\mathbb{Z}}\, [\varphi \mapsto v]\, v_0) \to_\beta \mathtt{comp}^i P(\mathtt{hfill}^i_{\mathbb{Z}}\, [\varphi \mapsto v]\, v_0\, i)\, [\varphi \mapsto f(v)]\, (f(v_0))$$

eq の義務は、Lemma 60 を線 $P(\mathtt{eq}\, m\, n\, p\, q\, e\, i)$ に適用すれば埋まる。trunc の義務には 2 つの補題を重ねて用いる。各 r で線 $s \mapsto P(\mathtt{trunc}\, x\, y\, p\, q\, r\, s)$ の $s = 1$ の型は $P(y)$ で

あり、これは命題だから Lemma 55 により集合である。よって Lemma 61 から、r を動かした族

$$B \overset{def}{\equiv} \mathrm{PathP}^{s}\, P(\mathtt{trunc}\, x\, y\, p\, q\, r\, s)\, (f\, x)\, (f\, y)$$

は各点で命題となる。この線 B に Lemma 60 を適用すれば、端点 $\langle s\rangle\, f(p\, s)$, $\langle s\rangle\, f(q\, s)$ を結ぶ依存 path、すなわち d_{trunc} が得られる（機械検証は IntCore の elimProp を参照）。

複数引数の版も同様である：たとえば 3 変数の性質 $(x, y, w : \mathbb{Z}) \to P(x, y, w)$ は、Π 型が命題性を保つこと（Lemma 63）により $\lambda x.\, (y, w : \mathbb{Z}) \to P$ が各点で命題となるから、Lemma 62 を入れ子に適用して、point 構成子の組の場合だけに帰着する。

Lemma 63 (Π Preserves h-Levels)**.** $B : A \to \mathrm{type}$ の各 $B(x)$ が命題（それぞれ集合）ならば、$(x : A) \to B(x)$ も命題（それぞれ集合）である。

確認：命題の場合、$f, g : (x : A) \to B(x)$ に対し $\langle i\rangle\, \lambda x.h_x\, (f\, x)\, (g\, x)\, i$（$h_x$ は $B(x)$ の命題性）が f から g への path を与える（Section 2.2 の funext と同じ形である）。集合の場合も、path どうしを結ぶ正方形を各点の正方形から同じ仕方で束ねればよい。

■$\mathbb{Z}$ 上の加法　自然数の加法 $m + n \overset{def}{\equiv} \mathtt{natrec}^{\lambda_.\mathbb{N}}_{m}(n, \lambda x.\lambda y.\mathtt{s}(y))$（第 1 引数上の再帰；$\mathtt{0} + n = n$、$\mathtt{s}(m) + n = \mathtt{s}(m + n)$ が判断的）を用いる。まず $\mathbb{N}$ 上の補題を揃える。以下、path の合成 $\cdot$ と反転 $(-)^{-1}$ は Section 3.3 のものである。

Lemma 64 (Lemmas on $\mathbb{N}$)**.** 任意の $m, n, k : \mathbb{N}$ に対して、次の path が natrec により構成できる（いずれも「motive を立てて基底と帰納段階を与える」標準形。基底はすべて判断的等値性により定値 path、帰納段階は $\langle i\rangle\, \mathtt{s}(p\, i)$ の形か、その既出 path との合成である）：

$$\begin{aligned}
&\mathrm{addSucc}(m, n) &&: \mathrm{Path}\, \mathbb{N}\, (m + \mathtt{s}(n))\, (\mathtt{s}(m + n))\\
&\mathrm{addZero}(m) &&: \mathrm{Path}\, \mathbb{N}\, (m + \mathtt{0})\, m\\
&\mathrm{assoc}_{\mathbb{N}}(m, n, k) &&: \mathrm{Path}\, \mathbb{N}\, ((m + n) + k)\, (m + (n + k))\\
&\mathrm{comm}_{\mathbb{N}}(m, n) &&: \mathrm{Path}\, \mathbb{N}\, (m + n)\, (n + m)
\end{aligned}$$

確認：addSucc・addZero・$\mathrm{assoc}_{\mathbb{N}}$ は、基底が判断的等値性により定値 path、帰納段階が $\langle i\rangle\, \mathtt{s}(p\, i)$（IH の path p を $\mathtt{s}$ に通したもの）で、端点は加法の計算規則により判断的に合う。$\mathrm{comm}_{\mathbb{N}}$ のみ合成を要する：基底は $\mathtt{0} + n = n$（判断的）と $n + \mathtt{0}$ を結ぶ $\mathrm{addZero}(n)^{-1}$、帰納段階は $\mathtt{s}(x) + n = \mathtt{s}(x + n)$（判断的）から出発して $\langle i\rangle\, \mathtt{s}(p\, i)$ で $\mathtt{s}(n + x)$ に至り、$\mathrm{addSucc}(n, x)^{-1}$ で $n + \mathtt{s}(x)$ に至る—natrec の帰納段階の中で hcomp による合成を使う最初の例である。

Lemma 65 (Shuffling Lemmas)**.** 任意の $a, b, c, d : \mathbb{N}$ に対して、$\mathrm{assoc}_{\mathbb{N}}$・$\mathrm{comm}_{\mathbb{N}}$ の合成として次の path が構成できる：

$$\begin{aligned}
&\mathrm{ichg}(a, b, c, d) &&: \mathrm{Path}\, \mathbb{N}\, ((a + b) + (c + d))\, ((a + c) + (b + d))\\
&\mathrm{swap}(a, b, c) &&: \mathrm{Path}\, \mathbb{N}\, ((a + b) + c)\, ((a + c) + b)
\end{aligned}$$

確認（ichg）：$(a+b)+(c+d) \to a+(b+(c+d)) \to a+((b+c)+d) \to a+((c+b)+d) \to a+(c+(b+d)) \to (a+c)+(b+d)$ と、結合律 4 回と交換律 1 回（に $\langle i\rangle\, a + (-i)$ 型の合同を添えたもの）の合成である。swap も同様。

well-definedness の中身は、古典的構成で「同値性にもとづく置換」として証明される補題と同じ計算である：

Lemma 66 (Compatibility of Addition)**.** 任意の $m, n, p, q, p', q', m', n' : \mathbb{N}$ に対して：

$$e : \mathrm{Path}\,\mathbb{N}\,(p + q')\,(p' + q) \text{ から}$$
$$\mathrm{Path}\,\mathbb{N}\,((m + p) + (n + q'))\,((m + p') + (n + q))$$
$$e : \mathrm{Path}\,\mathbb{N}\,(m + n')\,(m' + n) \text{ から}$$
$$\mathrm{Path}\,\mathbb{N}\,((m + p) + (n' + q))\,((m' + p) + (n + q))$$

が構成できる。

確認（第 1 式）：

$$(m + p) + (n + q') \xrightarrow{\text{ichg}} (m + n) + (p + q') \xrightarrow{\langle i\rangle\,(m+n)+e\,i} (m + n) + (p' + q)$$
$$\xrightarrow{\text{ichg}^{-1}} (m + p') + (n + q)$$

の合成である。第 2 式も同様（ichg で e の張れる位置に組み替え、合同で通し、組み替えを戻す）。

$\mathbb{Z}$ 上の加法は、recursion principle を 2 段重ねて定義する。外側は第 1 引数上の除去で、行き先は関数型 $\mathbb{Z} \to \mathbb{Z}$（Lemma 63 と `trunc` により集合）である。point 構成子どうしの場合の値は、差分の成分ごとの加法とする：

$$[\langle m, n\rangle] +_{\mathbb{Z}} [\langle p, q\rangle] \stackrel{def}{\equiv} [\langle m + p, n + q\rangle]$$

残る義務は well-definedness、すなわち d_{eq} の 2 つ（内側＝第 2 引数の `eq`、外側＝第 1 引数の `eq`）であり、いずれも Lemma 66 の path を構成子 `eq` に渡すだけである：たとえば内側は、$e : \mathrm{Path}\,\mathbb{N}\,(p + q')\,(p' + q)$ に対して

$$\langle i\rangle\ \mathtt{eq}\,(m{+}p)\,(n{+}q)\,(m{+}p')\,(n{+}q')\,(\text{Lemma 66 第 1 式の path})\,i$$

が $[\langle m + p, n + q\rangle]$ から $[\langle m + p', n + q'\rangle]$ への求める path を与える（外側は第 2 式と関数外延性による。`trunc` の義務は h_C が処理する）。

Remark 67 (Congruence Comes for Free)**.** 定義がこの義務を果たして**しまえば**、あとの合同性はすべて無償である：$u : \mathrm{Path}\,\mathbb{Z}\,x\,x'$ と $v : \mathrm{Path}\,\mathbb{Z}\,y\,y'$ から

$$\langle i\rangle\,(u\,i) +_{\mathbb{Z}} (v\,i) \ :\ \mathrm{Path}\,\mathbb{Z}\,(x +_{\mathbb{Z}} y)\,(x' +_{\mathbb{Z}} y')$$

が直ちに得られる。古典的構成で定理として証明された「同値性にもとづく置換」の定理が、「すべての関数は path を保つ」という path 抽象の一般原理の instance になるのである。well-definedness の証明が定義時の一度きりで済み、使用のたびの置換が自動化される—これが商を型に内部化することの利得である。

4.6 Basic Theorems on $\mathbb{Z}$

本節では、古典的な商集合による構成で示される $\mathbb{Z}$ の基本定理たちを、Path 型の構成として証明する。示すべきものはすべて「$\mathbb{Z}$ の元どうしを結ぶ path の存在」であり、

`trunc` により $\mathbb{Z}$ は集合、したがって path 型 $\mathrm{Path}\,\mathbb{Z}\,u\,v$ はつねに**命題**である。よって Lemma 62 が一律に適用でき、どの定理も「point 構成子の場合を、成分ごとの $\mathbb{N}$ の補題で示す」ことに帰着する。`eq`・`trunc`・hcomp の場合分けは以後いっさい現れない。

Definition 68 (Negation and Subtraction on $\mathbb{Z}$)**.** $\mathtt{neg} : \mathbb{Z} \to \mathbb{Z}$ を recursion principle により $\mathtt{neg}([\langle m, n\rangle]) \overset{def}{\equiv} [\langle n, m\rangle]$ で定義する。well-definedness の義務は、$e : \mathrm{Path}\,\mathbb{N}\,(m+q)\,(p+n)$ から $\mathrm{Path}\,\mathbb{N}\,(n+p)\,(q+m)$ を作ること（成分の入れ替え）であり、$\mathrm{comm}_{\mathbb{N}} \cdot e^{-1} \cdot \mathrm{comm}_{\mathbb{N}}$ の形の合成で得られる。減法は $y -_{\mathbb{Z}} x \overset{def}{\equiv} y +_{\mathbb{Z}} \mathtt{neg}(x)$ と定義する。

Theorem 69 (Double Negation)**.** 任意の $x : \mathbb{Z}$ に対して $\mathrm{Path}\,\mathbb{Z}\,(\mathtt{neg}(\mathtt{neg}(x)))\,x$ が成り立つ。

証明：Lemma 62 により point の場合だけでよく、そこでは $\mathtt{neg}(\mathtt{neg}([\langle m, n\rangle])) = [\langle m, n\rangle]$ が判断的に成り立つから定値 path で足りる。 □

Theorem 70 (Unit Laws)**.** $0_{\mathbb{Z}} \overset{def}{\equiv} [\langle \mathtt{0}, \mathtt{0}\rangle]$ とおく。任意の $x : \mathbb{Z}$ に対して $\mathrm{Path}\,\mathbb{Z}\,(0_{\mathbb{Z}} +_{\mathbb{Z}} x)\,x$ と $\mathrm{Path}\,\mathbb{Z}\,(x +_{\mathbb{Z}} 0_{\mathbb{Z}})\,x$ がともに成り立つ。

証明：point の場合、左単位元律は $0_{\mathbb{Z}} +_{\mathbb{Z}} [\langle m, n\rangle] = [\langle \mathtt{0} + m, \mathtt{0} + n\rangle] = [\langle m, n\rangle]$ が判断的に成り立つから定値 path。右単位元律は $a + \mathtt{0} = a$ が判断的でないから、addZero を成分ごとに通した

$$\langle i\rangle\,[\langle \mathrm{addZero}(m)\,i, \mathrm{addZero}(n)\,i\rangle]$$

による。加法の定義（第 1 引数上の再帰）に由来する判断的／命題的の非対称が、そのまま左右の証明の軽重に現れている。 □

Theorem 71 (Additive Inverse)**.** 任意の $x : \mathbb{Z}$ に対して $\mathrm{Path}\,\mathbb{Z}\,(x +_{\mathbb{Z}} \mathtt{neg}(x))\,0_{\mathbb{Z}}$ が成り立つ。

証明：point の場合、$[\langle m, n\rangle] +_{\mathbb{Z}} \mathtt{neg}([\langle m, n\rangle]) = [\langle m + n, n + m\rangle]$ であり、これは差分として $\langle \mathtt{0}, \mathtt{0}\rangle$ と同値である：witness は $(m + n) + \mathtt{0} = \mathtt{0} + (n + m)$ の path、すなわち $\mathrm{addZero}(m{+}n) \cdot \mathrm{comm}_{\mathbb{N}}(m, n)$ である。よって

$$\langle i\rangle\,\mathtt{eq}\,(m{+}n)\,(n{+}m)\,\mathtt{0}\,\mathtt{0}\,(\mathrm{addZero}(m{+}n) \cdot \mathrm{comm}_{\mathbb{N}}(m, n))\,i$$

が求める path を与える。構成子 `eq` が定義の外で仕事をする最初の例である。 □

Theorem 72 (Associativity of $+_{\mathbb{Z}}$)**.** 任意の $x, y, w : \mathbb{Z}$ に対して $\mathrm{Path}\,\mathbb{Z}\,((x +_{\mathbb{Z}} y) +_{\mathbb{Z}} w)\,(x +_{\mathbb{Z}} (y +_{\mathbb{Z}} w))$ が成り立つ。

証明：3 変数の性質だから、Lemma 62（入れ子適用）により point 構成子の組の場合だけでよい。そこでは両辺が $[\langle (m + p) + u, (n + q) + v\rangle]$ と $[\langle m + (p + u), n + (q + v)\rangle]$ に判断的に計算され、

$$\langle i\rangle\,[\langle \mathrm{assoc}_{\mathbb{N}}(m, p, u)\,i, \mathrm{assoc}_{\mathbb{N}}(n, q, v)\,i\rangle]$$

が求める path である。 □

Exercise 73. 加法交換律 $\mathrm{Path}\,\mathbb{Z}\,(x +_{\mathbb{Z}} y)\,(y +_{\mathbb{Z}} x)$ を証明せよ。（ヒント：Theorem 72 と同じ形。point の場合は $\mathrm{comm}_{\mathbb{N}}$ を成分ごとに。）

Theorem 74 (Cancellation)**.** 任意の $x, y, w : \mathbb{Z}$ に対して、$\mathrm{Path}\,\mathbb{Z}\,(x +_{\mathbb{Z}} w)\,(y +_{\mathbb{Z}} w)$ ならば $\mathrm{Path}\,\mathbb{Z}\,x\,y$ が成り立つ。

証明：こちらは除去を使わず、既に得た法則の path の合成だけで済む（古典的な群論的等式変形の翻訳である）。$h : \mathrm{Path}\,\mathbb{Z}\,(x +_{\mathbb{Z}} w)\,(y +_{\mathbb{Z}} w)$ とすると、次の連鎖が構成できる。ただし各行の $=$ は判断的等値性ではなく、右欄の根拠が与える $\mathbb{Z}$ の **path** を表す：

$$
\begin{array}{rcll}
x & = & x +_{\mathbb{Z}} 0_{\mathbb{Z}} & \text{Theorem 70（右）の反転} \\
 & = & x +_{\mathbb{Z}} (w +_{\mathbb{Z}} \mathtt{neg}(w)) & \text{Theorem 71 の反転を合同で} \\
 & = & (x +_{\mathbb{Z}} w) +_{\mathbb{Z}} \mathtt{neg}(w) & \text{Theorem 72 の反転} \\
 & = & (y +_{\mathbb{Z}} w) +_{\mathbb{Z}} \mathtt{neg}(w) & h \text{ を合同で（Remark 67）} \\
 & = & y +_{\mathbb{Z}} (w +_{\mathbb{Z}} \mathtt{neg}(w)) & \text{Theorem 72} \\
 & = & y +_{\mathbb{Z}} 0_{\mathbb{Z}} & \text{Theorem 71 を合同で} \\
 & = & y & \text{Theorem 70（右）}
\end{array}
$$

の hcomp による合成が求める path である。 □

Exercise 75. 任意の $a, b : \mathbb{Z}$ に対して、$x \overset{def}{\equiv} a -_{\mathbb{Z}} b$ が方程式 $x +_{\mathbb{Z}} b = a$ の解であること、すなわち $\mathrm{Path}\,\mathbb{Z}\,((a -_{\mathbb{Z}} b) +_{\mathbb{Z}} b)\,a$ を、Theorem 72・Theorem 70・Theorem 71・Exercise 73 を組み合わせた path の合成のみで構成せよ。解の一意性は Theorem 74 そのものである。

以上のとおり、eq + trunc 版では、すべての法則が「Lemma 62 +成分ごとの $\mathbb{N}$ の補題」か「既出の法則の合成」のどちらかの一様な形をとる。乗法とその法則（結合律・交換律・分配律・消去律）も、well-definedness の補題（同値性にもとづく置換の乗法側）を Lemma 66 と同様に用意すれば、まったく同じ手筋で構成できる。同じ設計は $\mathbb{Q}$（$\mathbb{Z}$ の対の商）や $\mathbb{R}$（Cauchy 列の商）にも反復できる—商をとるたびに eq と trunc を書き、well-definedness を d_{eq} に払う、という一様な手続きである。これと対照的に、path 構成子を生成元 1 本に絞って truncation を消す別の設計（そこでは正方形の証明義務を手で充填することになる）を、Section 5.1 で与える。

4.7 The Normal Form Theorem

`eq` と `trunc` は、どちらも path を**増やす**方向の構成子である。増やしすぎてすべてが潰れていないこと—たとえば $[\langle 1, \mathtt{0}\rangle] = 0_{\mathbb{Z}}$ になっていないこと—は、自明ではなく、証明を要する。本節では $\mathbb{Z}$ が標準形の型と同値であることを示し、そこから等値性の決定可能性と、商の効果性（effectivity）—$[\langle m, n\rangle] = [\langle p, q\rangle]$ ならば $m + q = p + n$、すなわち `eq` の**逆**—を導く。古典的構成で「任意の整数は $[\langle n, \mathtt{0}\rangle]$ か $[\langle \mathtt{0}, m\rangle]$（$m \neq 0$）のいずれか一方の標準形で表される」と述べられた事実の、cubical における対応物である。

標準形の型を選言型により

$$\mathrm{Int} \overset{def}{\equiv} \mathbb{N} + \mathbb{N}$$

と定め、$\iota_1(n)$ を整数 n、$\iota_2(m)$ を整数 $-(m+1)$（すなわち $-1, -2, \ldots$）と読む。正規化関数 $\mathrm{norm} : \mathbb{N} \to \mathbb{N} \to \mathrm{Int}$ を、**第 2 引数上の**再帰により、$\mathbb{N} \to \mathrm{Int}$ を target として

定義する：

$$\mathrm{norm}_{\mathtt{0}} \overset{def}{\equiv} \lambda a.\iota_1(a)$$
$$\mathrm{norm}_{\mathtt{s}(b)} \overset{def}{\equiv} \lambda a.\mathtt{natrec}_a^{\lambda_.\mathrm{Int}}(\iota_2(b), \lambda a'.\lambda_.\mathrm{norm}_b(a'))$$
$$\mathrm{norm}(a,b) \overset{def}{\equiv} \mathrm{norm}_b(a)$$

この符号化により、次の 3 つの等式がいずれも**判断的に**成り立つ（a が変数のままでも成り立つことが要点である）：

$$\mathrm{norm}(a,\mathtt{0}) = \iota_1(a) \qquad \mathrm{norm}(\mathtt{0},\mathtt{s}(b)) = \iota_2(b) \qquad \mathrm{norm}(\mathtt{s}(a),\mathtt{s}(b)) = \mathrm{norm}(a,b)$$

$\mathrm{toInt} : \mathbb{Z} \to \mathrm{Int}$ を recursion principle で定義したいが、そのためには行き先 Int が**集合であること**が要る。これは次の一般的事実による：*Hedberg* の定理—等値性が決定可能な型は集合である（The Univalent Foundations Program 2013 の定理 7.2.5。本書では証明は引用に譲る）。$\mathbb{N}$ の等値性は natrec により決定可能であり、したがって $\mathrm{Int} = \mathbb{N} + \mathbb{N}$ の等値性も決定可能だから、$\mathbb{N}$ も Int も集合である。なお Chapter 1 で「MLTT では UIP は導出できない」ことを示したが、それは**一様には**導出できないという意味であって、決定可能な等値性をもつ個々の型については、このように UIP（集合性）が導出できる。

well-definedness の鍵は次の補題である。

Lemma 76. 任意の $k, m, n : \mathbb{N}$ に対して $\mathrm{Path}\,\mathrm{Int}\,(\mathrm{norm}(k + m, k + n))\,(\mathrm{norm}(m, n))$ が構成できる。

確認：k 上の natrec による。基底は $\mathtt{0} + m = m$（判断的）により定値 path。帰納段階では $\mathrm{norm}(\mathtt{s}(k) + m,\ \mathtt{s}(k) + n) = \mathrm{norm}(\mathtt{s}(k + m),\ \mathtt{s}(k + n)) = \mathrm{norm}(k + m,\ k + n)$ が**すべて判断的**に成り立つ（第 1 の等号は加法の計算規則、第 2 は norm の第 3 等式）から、帰納法の仮定の path がそのまま（conversion を経て）使える。古典的構成の「$\langle m, n\rangle \sim_d \langle k + m, k + n\rangle$」にあたる事実が、標準形の水準では判断的計算の繰り返しになっている。

Lemma 77 (norm Respects $\sim_d$)**.** $e : \mathrm{Path}\,\mathbb{N}\,(m + q)\,(p + n)$ から $\mathrm{Path}\,\mathrm{Int}\,(\mathrm{norm}(m, n))\,(\mathrm{norm}(p, q))$ が構成できる。

確認：

$$\mathrm{norm}(m,n) \xleftarrow{Lemma\ 76} \mathrm{norm}(q+m,\ q+n) \xrightarrow{\text{成分の path}} \mathrm{norm}(n+p,\ n+q) \xrightarrow{Lemma\ 76} \mathrm{norm}(p,q)$$

の合成である。中央の path は、第 1 成分 $q+m \to m+q \to p+n \to n+p$（$\mathrm{comm}_{\mathbb{N}} \cdot e \cdot \mathrm{comm}_{\mathbb{N}}$）と第 2 成分 $q+n \to n+q$（$\mathrm{comm}_{\mathbb{N}}$）を norm に通した合同 $\langle i\rangle\,\mathrm{norm}(\cdots i, \cdots i)$ である。

これで往復の写像が定義できる：

$$\mathrm{toInt} : \mathbb{Z} \to \mathrm{Int} \qquad \mathrm{toInt}([\langle m, n\rangle]) \overset{def}{\equiv} \mathrm{norm}(m, n) \quad (d_{\mathrm{eq}} \text{ は Lemma 77})$$
$$\mathrm{fromInt} : \mathrm{Int} \to \mathbb{Z} \quad \mathrm{fromInt}(\iota_1(n)) \overset{def}{\equiv} [\langle n, \mathtt{0}\rangle] \qquad \mathrm{fromInt}(\iota_2(m)) \overset{def}{\equiv} [\langle \mathtt{0}, \mathtt{s}(m)\rangle]$$

Theorem 78 (Normal Form)**.** toInt と fromInt は互いに逆であり、$\mathbb{Z} \simeq \mathrm{Int}$ が成り立つ。

証明：

(i) $\mathrm{toInt} \circ \mathrm{fromInt}$ の側は Int 上の場合分けによる。$v = \iota_1(n)$ のとき $\mathrm{toInt}([\langle n, \mathtt{0}\rangle]) = \mathrm{norm}(n, \mathtt{0}) = \iota_1(n)$、$v = \iota_2(m)$ のとき $\mathrm{toInt}([\langle \mathtt{0}, \mathtt{s}(m)\rangle]) = \mathrm{norm}(\mathtt{0}, \mathtt{s}(m)) = \iota_2(m)$ がいずれも判断的に成り立つ（norm の第 1・第 2 等式）。よって定値 path でよい。

(ii) $\mathrm{fromInt} \circ \mathrm{toInt}$ の側は、ゴール $\mathrm{Path}\,\mathbb{Z}\,(\mathrm{fromInt}(\mathrm{toInt}(z)))\,z$ が（trunc の与える集合性により）命題だから、Lemma 62 が使える：point の場合だけでよい。$m_{a,b} : \mathrm{Path}\,\mathbb{Z}\,(\mathrm{fromInt}(\mathrm{norm}(a,b)))\,[\langle a, b\rangle]$ を、b そして a の二重帰納法で構成する：

$$
\begin{aligned}
m_{a,\mathtt{0}} &\overset{def}{\equiv} 1_{[\langle a, \mathtt{0}\rangle]} \\
m_{\mathtt{0},\mathtt{s}(b)} &\overset{def}{\equiv} 1_{[\langle \mathtt{0}, \mathtt{s}(b)\rangle]} \\
m_{\mathtt{s}(a),\mathtt{s}(b)} &\overset{def}{\equiv} m_{a,b} \cdot \langle i\rangle\, \mathtt{eq}\, a\, b\, (\mathtt{s}(a))\, (\mathtt{s}(b))\, (\mathrm{addSucc}(a,b))\, i
\end{aligned}
$$

最初の 2 つは norm の第 1・第 2 等式と fromInt の定義から判断的。第 3 の場合は、$\mathrm{norm}(\mathtt{s}(a), \mathtt{s}(b)) = \mathrm{norm}(a, b)$（判断的）により左辺の型が

$$\mathrm{Path}\,\mathbb{Z}\,(\mathrm{fromInt}(\mathrm{norm}(a,b)))\,[\langle \mathtt{s}(a), \mathtt{s}(b)\rangle]$$

となるから、$m_{a,b}$ の終点 $[\langle a, b\rangle]$ から $[\langle \mathtt{s}(a), \mathtt{s}(b)\rangle]$ への eq の path（witness は $a + \mathtt{s}(b) = \mathtt{s}(a) + b$、すなわち $\mathrm{addSucc}(a,b)$；右辺は判断的に $\mathtt{s}(a+b)$）を合成すればよい。 □

cancel 版（Section 5.1）ではこの（ii）の path 構成子の場合に、合成の充填（hfill）による正方形を手で与える必要が生じる。ここでは trunc と Lemma 62 がその義務を自動的に処理している—「集合性を公理として買った」ことの見返りである。

この定理から直ちに従うことを挙げる。

- $\mathbb{N}$ の等値性の決定可能性から Int の等値性は決定可能であり、同値 $\mathbb{Z} \simeq \mathrm{Int}$ によりこれを $\mathbb{Z}$ へ移送できる：$\mathbb{Z}$ の等値性は決定可能である。
- 任意の $z : \mathbb{Z}$ は $[\langle n, \mathtt{0}\rangle]$ か $[\langle \mathtt{0}, \mathtt{s}(m)\rangle]$ のいずれか一方と path で結ばれる。これが古典版の「$\mathbb{Z}$ を二分する」（標準形の存在と一意性）に対応する。正負と 0 による三分法は、これに Int 側で $\iota_1(\mathtt{0})$ を分離して得られる。
- 埋め込み $n \longmapsto [\langle n, \mathtt{0}\rangle]$ は単射である（次の効果性から従う：$[\langle n, \mathtt{0}\rangle] = [\langle n', \mathtt{0}\rangle]$ ならば $n + \mathtt{0} = n' + \mathtt{0}$、addZero で $n = n'$）。

■商の効果性 最後に、eq の逆—path から同値関係の witness の回復—を示す。準備として $\mathbb{N}$ の補題を 2 つ用意する。

Lemma 79 (Cancellation on $\mathbb{N}$)**.** 前者関数 pd を natrec により $\mathrm{pd}(\mathtt{0}) \overset{def}{\equiv} \mathtt{0}$, $\mathrm{pd}(\mathtt{s}(n)) \overset{def}{\equiv} n$ と定義すると、$h : \mathrm{Path}\,\mathbb{N}\,(\mathtt{s}(m))\,(\mathtt{s}(n))$ から $\langle i\rangle\, \mathrm{pd}(h\, i) : \mathrm{Path}\,\mathbb{N}\, m\, n$ が得られる（s の単射性）。これを用いて、b 上の帰納法により：$\mathrm{Path}\,\mathbb{N}\,(x + b)\,(y + b)$ ならば $\mathrm{Path}\,\mathbb{N}\, x\, y$。（基底は addZero で両辺を整えるだけ。帰納段階は addSucc で両辺を $\mathtt{s}(x+b)$, $\mathtt{s}(y+b)$ に整え、s の単射性で剥がして帰納法の仮定に渡す。）

Lemma 80 (Soundness of norm)**.** 成分の読み出し $\mathrm{rb}_1, \mathrm{rb}_2 : \mathrm{Int} \to \mathbb{N}$ を

$$\mathrm{rb}_1(\iota_1(k)) \overset{def}{\equiv} k \quad \mathrm{rb}_1(\iota_2(k)) \overset{def}{\equiv} \mathtt{0} \qquad \mathrm{rb}_2(\iota_1(k)) \overset{def}{\equiv} \mathtt{0} \quad \mathrm{rb}_2(\iota_2(k)) \overset{def}{\equiv} \mathtt{s}(k)$$

と定めると、任意の $m, n : \mathbb{N}$ に対して

$$\mathrm{Path}\,\mathbb{N}\,(m + \mathrm{rb}_2(\mathrm{norm}(m,n)))\,(\mathrm{rb}_1(\mathrm{norm}(m,n)) + n)$$

が構成できる。すなわち、$\mathrm{norm}(m,n)$ の表す差分は $\langle m, n\rangle$ と同値である。

確認：(ii) の $m_{a,b}$ と同じ 3 分割の二重帰納法による。$(m, \mathtt{0})$ と $(\mathtt{0}, \mathtt{s}(b))$ の場合は norm の第 1・第 2 等式により両辺が判断的に一致する（定値 path）。$(\mathtt{s}(a), \mathtt{s}(b))$ の場合は norm の第 3 等式により読み出しが変わらないから、帰納法の仮定 p に対して $\langle i\rangle\, \mathtt{s}(p\,i) \cdot \mathrm{addSucc}(\mathrm{rb}_1, b)^{-1}$ の形で両辺の $\mathtt{s}$ を整えればよい。

Theorem 81 (Effectivity)**.** $h : \mathrm{Path}\,\mathbb{Z}\,[\langle m,n\rangle]\,[\langle p,q\rangle]$ から $\mathrm{Path}\,\mathbb{N}\,(m+q)\,(p+n)$ が構成できる。

証明：$\bar{h} \overset{def}{\equiv} \langle i\rangle\, \mathrm{toInt}(h\,i) : \mathrm{Path}\,\mathrm{Int}\,(\mathrm{norm}(m,n))\,(\mathrm{norm}(p,q))$ をとり、$a \overset{def}{\equiv} \mathrm{rb}_1(\mathrm{norm}(m,n))$, $b \overset{def}{\equiv} \mathrm{rb}_2(\mathrm{norm}(m,n))$ とおく。Lemma 80 が $e_1 : \mathrm{Path}\,\mathbb{N}\,(m+b)\,(a+n)$ を与え、$\mathrm{norm}(p,q)$ 側の soundness を $\bar{h}$ に rb_1・rb_2 を通した合同で書き換えると $e_2 : \mathrm{Path}\,\mathbb{N}\,(p+b)\,(a+q)$ を得る。あとは

$$(m+q)+b \;\to\; (m+b)+q \xrightarrow{e_1} (a+n)+q \;\to\; (a+q)+n \xrightarrow{e_2^{-1}} (p+b)+n \;\to\; (p+n)+b$$

（無印の矢印は Lemma 65 の swap。また $e_1 \cdot e_2^{-1}$ の段は、正確にはそれらを加数の下で用いる合同 $\langle i\rangle\,(e_1\,i)+q \cdot \langle i\rangle\,(e_2^{-1}\,i)+n$ である）と合成して $\mathrm{Path}\,\mathbb{N}\,((m+q)+b)\,((p+n)+b)$ を得て、Lemma 79 で b を消去すればよい。 □

eq（同値なら等しい）と Theorem 81（等しいなら同値）を併せて、$\mathrm{Path}\,\mathbb{Z}\,[\langle m,n\rangle]\,[\langle p,q\rangle]$ と $\mathrm{Path}\,\mathbb{N}\,(m+q)\,(p+n)$ が論理的に同値であることが確定した。商は「ちょうど $\sim_d$ の分だけ」潰れており、それ以上でも以下でもない。

古典版の正規形定理の証明が $g \circ f = id$ と $f \circ g = id$ の 2 つの計算からなっていたことを思い出そう。cubical 版でも骨格は同じであり、(i) の側は判断的計算だけで済む一方、(ii) の側にのみ eq の path が必要になる。これは古典版で $\sim_d$ の計算を要したのが $f \circ g$ の側だけであったことに正確に対応している。

Remark 82 (Predicates on $\mathbb{Z}$)**.** 古典的構成はこの後、正値性 $\mathbb{Z}^+$ と順序 $<$ を定義し、三分法・全順序性・整列性へ進む。その出発点は述語 $P : \mathbb{Z} \to \mathrm{type}_i$ であるが、**述語を $\mathbb{Z}$ の除去で直接定義することはできない**。eq の義務は型と型を結ぶ path を要求するが、これは Chapter 5 で導入する univalence（ua）で満たせる。障害は trunc の義務である：除去には行き先が集合であることが要るが、univalence のもとで universe type_i は集合では**ない**[*4]。

この障害を避ける道は 3 つある。(a) は行き先を狭める道であり、(b)・(c) は $\mathbb{Z}$ をいったん別の型へ写して、そちらで述語を定義する道である。

[*4] たとえば 2 元の列挙型は恒等と入れ替えの 2 つの自己同値をもち、ua がそれぞれを path にする。両者に沿った移送は異なる値を返すから、この 2 本は Path で結べない。

(a) 行き先を「命題の型」$(X : \mathrm{type}_i) \times \mathbf{isProp}(X)$ に絞る。これは集合になることが知られている（The Univalent Foundations Program 2013 の定理 7.1.11）。値が命題である述語なら、この範囲で足りる。
(b) Int の上で場合分けにより述語を定義し、Theorem 78 の toInt と合成する。Int は path 構成子をもたないので、義務は生じず、univalence も要らない。
(c) truncation をもたない Section 5.1 の cancel 版 $\mathbb{Z}_c$ の上で述語を定義し、同値 $\mathbb{Z} \simeq \mathbb{Z}_c$ と合成する。行き先の条件は消えるが、`cancel` の義務に ua を用いる。

最も手軽なのは (b) であるが、本書は Chapter 5 で (c) を実演する。ua による型値再帰の実例になるからである。

4.8 Free Monoids and Groups

$\mathbb{Z}$ の `eq` は、**特定の同値関係**を path 構成子として型に焼き込んだ。本節では最後の一般化として、**演算とその等式公理**を焼き込む—すなわち代数構造の自由代数（free algebra）を高次帰納型として構成する。ここで初めて**再帰的な point 構成子**（定義中の型の元から新しい元を作る「演算」）が登場し、Section 4.1 のスキーマに現れうる構成子の種類が、本章の実例ですべて出揃うことになる。

Definition 83 (The Free Monoid)**.** 型 $\Gamma \vdash A : \mathrm{type}$ をパラメータとして、高次帰納型 $\mathrm{FreeMon}(A)$ は次の構成子により定義される：

$$\frac{\Gamma \ \mathit{context}}{\Gamma \vdash \mathtt{e} : \mathrm{FreeMon}(A)} \qquad \frac{\Gamma \vdash a : A}{\Gamma \vdash \mathtt{gen}\, a : \mathrm{FreeMon}(A)} \qquad \frac{\Gamma \vdash x, y : \mathrm{FreeMon}(A)}{\Gamma \vdash x \odot y : \mathrm{FreeMon}(A)}$$

$$\frac{\Gamma \vdash x : \mathrm{FreeMon}(A) \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \mathtt{unitL}\, x\, r : \mathrm{FreeMon}(A)} \qquad \frac{\Gamma \vdash x : \mathrm{FreeMon}(A) \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \mathtt{unitR}\, x\, r : \mathrm{FreeMon}(A)}$$

$$\frac{\Gamma \vdash x, y, z : \mathrm{FreeMon}(A) \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \mathtt{assoc}\, x\, y\, z\, r : \mathrm{FreeMon}(A)}$$

境界の等値性判断は

$$\begin{aligned} &\mathtt{unitL}\, x\, 0 =_\partial \mathtt{e} \odot x && \mathtt{unitL}\, x\, 1 =_\partial x \\ &\mathtt{unitR}\, x\, 0 =_\partial x \odot \mathtt{e} && \mathtt{unitR}\, x\, 1 =_\partial x \\ &\mathtt{assoc}\, x\, y\, z\, 0 =_\partial (x \odot y) \odot z && \mathtt{assoc}\, x\, y\, z\, 1 =_\partial x \odot (y \odot z) \end{aligned}$$

であり、さらに Definition 57 と同形の set truncation 構成子 `trunc`（と hcomp 構成子）を定義に含める。

point 構成子 $\odot$ が $\mathrm{FreeMon}(A)$ 自身の元を 2 つ引数にとること、そして**モノイドの等式公理がそのまま path 構成子になっている**ことに注目してほしい。`eq` が「商」を焼き込んだのと同じ手で、ここでは「演算の法則」を焼き込んでいる。`trunc` が省けないことも $\mathbb{Z}$ と同様である：たとえば `unitL e` と `unitR e` は、どちらも $\mathtt{e} \odot \mathtt{e}$ から e への辺であり、平行辺が閉路を作る。また $\mathrm{FreeMon}(A)$ は本章で初めての**パラメータをもつ**高次帰納型であり、線 A が動く場合の transp は Section 4.1 のとおり構成子ごとに与えられる（`gen` の引数を線に沿って運ぶ形になる）。以下では A を固定して用いるので、transp が働く場面はない。

■除去規則＝普遍性 C を集合とする。関数 $f : \mathrm{FreeMon}(A) \to C$ を recursion principle で定義するために与えるべきデータを、構成子ごとに書き出すと：

$$c_e : C \qquad \circledast : C \to C \to C \qquad d : A \to C$$

および、$\circledast$ が c_e を単位元とするモノイド法則を満たすことの証明（unitL・unitR・assoc の行き先となる path たち）である。β-規則は

$$f(\mathsf{e}) \to_\beta c_e \qquad f(\mathsf{gen}\, a) \to_\beta d\, a \qquad f(x \odot y) \to_\beta f(x) \circledast f(y)$$

（法則の path 構成子は与えた証明へ、trunc は C の集合性へ送られる）。つまり、除去が要求しているのは C **上のモノイド構造**と**写像** $d : A \to C$ の 2 つだけであり、結果の f は、β-規則のとおり e を c_e へ、$\odot$ を $\circledast$ へ写す**モノイド準同型**である。ここで、f の値のうちこちらが選べるのは、生成元の上の値 d だけである。$\mathrm{FreeMon}(A)$ の元はすべて e と $\mathsf{gen}\, a$ から $\odot$ で作られるから、それ以外の値は準同型であることから決まってしまう：たとえば $f(\mathsf{gen}\, a \odot \mathsf{gen}\, b)$ は $d\, a \circledast d\, b$ でしかありえない。すなわち d は、準同型へ**ちょうど 1 通りに拡張される**。これが自由モノイドの普遍性—「任意のモノイド C と写像 $d : A \to C$ に対し、$f(\mathsf{gen}\, a) = d\, a$ を満たす準同型 $f : \mathrm{FreeMon}(A) \to C$ がただ 1 つ存在する」—である。拡張の存在は除去規則**そのもの**として現れている。拡張の一意性は、$\mathbb{Z}$ のときと同じ論法（Lemma 62 の類似。point 構成子の場合だけ示せばよく、$\odot$ の場合は帰納法の仮定つき）で証明できる（Exercise 87）。なお、d を任意に選べるのは、生成元の間に関係がないからである。関係を課した代数（Remark 86）では、d はその関係を保たなければならない。

■1 生成自由モノイドは $\mathbb{N}$ である 生成元を 1 つ（$A \overset{def}{\equiv} \top$。その唯一の元を $*$ と書く）にすると、見慣れた型が現れる。

Theorem 84 (One-Generator Free Monoid)**.** $\mathrm{FreeMon}(\top) \simeq \mathbb{N}$ が成り立つ。

証明：往復の写像を作る。$\mathrm{toN} : \mathrm{FreeMon}(\top) \to \mathbb{N}$ は recursion principle による：データは $\mathbb{N}$ のモノイド構造と生成元の行き先、すなわち

$$c_e \overset{def}{\equiv} \mathtt{0} \qquad \circledast \overset{def}{\equiv} + \qquad d(*) \overset{def}{\equiv} \mathtt{s}(\mathtt{0})$$

であり、法則の行き先は、unitL が $\mathtt{0} + n = n$（判断的）により定値 path、unitR が addZero、assoc が $\mathrm{assoc}_{\mathbb{N}}$ である—**除去の義務が、Section 4.5 で用意した $\mathbb{N}$ の補題そのもの**になっている。逆向き $\mathrm{fromN} : \mathbb{N} \to \mathrm{FreeMon}(\top)$ は natrec で $\mathrm{fromN}(\mathtt{0}) \overset{def}{\equiv} \mathsf{e}$、$\mathrm{fromN}(\mathtt{s}(n)) \overset{def}{\equiv} \mathsf{gen}\, * \odot \mathrm{fromN}(n)$ と定める。

（i）$\mathrm{toN} \circ \mathrm{fromN} : n$ 上の帰納法。基底は判断的。帰納段階も $\mathrm{toN}(\mathsf{gen}\, * \odot \mathrm{fromN}(n)) = \mathtt{s}(\mathtt{0}) + \mathrm{toN}(\mathrm{fromN}(n)) = \mathtt{s}(\mathrm{toN}(\mathrm{fromN}(n)))$ がすべて判断的に計算されるから、帰納法の仮定を $\mathtt{s}$ に通すだけである。

（ii）$\mathrm{fromN} \circ \mathrm{toN}$：ゴールは（trunc の与える集合性により）命題だから、elimProp で point 構成子の場合だけでよい。e の場合は判断的。$\mathsf{gen}\, a$ の場合は $\mathrm{fromN}(\mathtt{s}(\mathtt{0})) = \mathsf{gen}\, * \odot \mathsf{e}$ だから、$\mathtt{unitR}$ と「$\top$ の任意の元は $*$ に path で結ばれる」こと（$\top$ の除去による）で $\mathsf{gen}\, a$ に至る。$x \odot y$ の場合は、fromN が加法を $\odot$ に写すこと：

$$\mathrm{Path}\,(\mathrm{FreeMon}(\top))\,(\mathrm{fromN}(m+n))\,(\mathrm{fromN}(m) \odot \mathrm{fromN}(n))$$

(m 上の帰納法。基底は `unitL` の反転、帰納段階は帰納法の仮定を $\mathtt{gen}\,* \odot (-)$ に通してから `assoc` の反転— **法則の構成子が実際に仕事をする**箇所である）を経由して、2 つの帰納法の仮定を $\odot$ の合同で貼り合わせればよい。 □

$\mathbb{N}$ とは「1 つの生成元から、法則以外の関係なしに生成されるモノイド」であった—natrec の正体が自由モノイドの普遍性だった、という見方がここで得られる。

■自由群と $\mathbb{Z}$ モノイドに逆元を加えると自由群になる。$\mathbf{FreeGrp}(A)$ は、Definition 83 の構成子に、再帰的 point 構成子 $\mathtt{inv}\,x$（$x : \mathbf{FreeGrp}(A)$）と、逆元律の path 構成子

$$\mathtt{invR}\,x\,0 =_\partial x \odot \mathtt{inv}\,x \qquad \mathtt{invR}\,x\,1 =_\partial \mathtt{e} \qquad \mathtt{invL}\,x\,0 =_\partial \mathtt{inv}\,x \odot x \qquad \mathtt{invL}\,x\,1 =_\partial \mathtt{e}$$

を加えたものである（trunc も同様）。recursion principle の要求は「C 上の**群**構造＋生成元の行き先」に変わる。

Theorem 85 (One-Generator Free Group)**.** $\mathbf{FreeGrp}(\top) \simeq \mathbb{Z}$ が成り立つ。

証明の構成：$\mathrm{toZ} : \mathbf{FreeGrp}(\top) \to \mathbb{Z}$ は recursion principle による。データは $\mathbb{Z}$ の群構造：

$$c_e \overset{def}{\equiv} 0_\mathbb{Z} \qquad \circledast \overset{def}{\equiv} +_\mathbb{Z} \qquad \mathtt{inv} \mapsto \mathtt{neg} \qquad d(*) \overset{def}{\equiv} [\langle \mathtt{s}(\mathtt{0}), \mathtt{0} \rangle]$$

であり、法則の行き先は **Section 4.6 で証明した定理群そのもの**である：unitL・unitR に Theorem 70、assoc に Theorem 72、invR に Theorem 71、invL には Theorem 71 と交換律（Exercise 73）の合成を充てる。あの節の法則たちは、「$\mathbb{Z}$ への自由群からの準同型を定義するためのデータ」でもあったのである。逆向き $\mathrm{fromZ} : \mathbb{Z} \to \mathbf{FreeGrp}(\top)$ は、$g \overset{def}{\equiv} \mathtt{gen}\,*$ とその冪（$x^0 \overset{def}{\equiv} \mathtt{e}$、$x^{\mathtt{s}(m)} \overset{def}{\equiv} x \odot x^m$）を用いて $\mathbb{Z}$ の recursion principle で $[\langle m, n \rangle] \longmapsto g^m \odot (\mathtt{inv}\,g)^n$ と定める。well-definedness（eq の場合）と往復の一致は、冪の準同型性 $x^{m+n} = x^m \odot x^n$ と $g \cdot \mathtt{inv}\,g$ の冪の可換性をもとにした補題の列に帰着する。組み立ては Theorem 84 (ii) と同じ elimProp の手筋であり、補題込みで Exercise 88 とする。 □

$\mathbf{FreeMon}(\top) \simeq \mathbb{N}$ と $\mathbf{FreeGrp}(\top) \simeq \mathbb{Z}$ を並べると、$\mathbb{N}$ から $\mathbb{Z}$ への移行とは「モノイドから群への移行（逆元の自由な添加）」だった、と要約できる。なお後者により $\mathbb{Z}$ は**第 3 の提示**（eq 版・cancel 版・自由群版）を得たことになり、3 つはいずれも univalence で結ばれる（Section 5.1 の移送の議論がそのまま適用できる）。

Remark 86 (Presentations of Algebras)**.** 本節の自由代数と、$\mathbb{Z}$ で用いた商（`eq` 方式）を組み合わせると、**生成元と関係による表示** $\langle$ 生成元 | 関係 $\rangle$ をもつ任意の代数が高次帰納型として書ける：自由代数の構成子に、関係の path 構成子（と trunc）を追加すればよい。また、一般の集合 A に対しては $\mathbf{FreeMon}(A)$ は A のリストの型と同値になることが知られている（リスト型は本書の基礎体系に含めていないため立ち入らない）。等式公理をもつ代数的理論の自由代数を高次帰納型（あるいはその一般化である quotient inductive type）として構成する枠組みについては、History and Further Reading の文献を参照。

Exercise 87. recursion principle の与える準同型の一意性を示せ：$f, f' : \mathbf{FreeMon}(A) \to C$ がともに同じデータ $(c_e, \circledast, d)$ に対する β-規則を（path として）満たすなら、任意の w で $\mathrm{Path}\,C\,(f\,w)\,(f'\,w)$ が成り立つ。（ヒント：C の集合性によりゴールは命題だから elimProp。$\odot$ の場合は帰納法の仮定 2 つを $\circledast$ の合同で貼り合わせる。）

Exercise 88. Theorem 85 の証明を、次の順に完成させよ。$h \overset{def}{\equiv} \mathtt{inv}\, g$ とおき、等号はすべて path を表す。

(1)（冪の準同型性）$x^{m+n} = x^m \odot x^n$ を m 上の帰納法で示せ（基底は `unitL`、帰納段階は `assoc`）。
(2)（可換性）$x \odot y = y \odot x$ ならば $x \odot y^n = y^n \odot x$ であることを、n 上の帰納法で示せ。これを 2 度用いて、$x \odot y = y \odot x$ ならば $x^m \odot y^n = y^n \odot x^m$ であることを導け。$g \odot h = \mathtt{e} = h \odot g$（`invR`・`invL`）だから、特に $g^m \odot h^n = h^n \odot g^m$ である。
(3)（相殺）$g^m \odot h^m = \mathtt{e}$ を m 上の帰納法で示せ（帰納段階は、(2) で g^m と h を入れ替えて $g \odot h$ を作る）。
(4)（共通項の消去）(1) 〜 (3) から $g^{m+k} \odot h^{n+k} = g^m \odot h^n$ を示せ。
(5)（fromZ の well-definedness）$e : \mathrm{Path}\,\mathbb{N}\,(m+q)\,(p+n)$ から $g^m \odot h^n = g^p \odot h^q$ を示せ（(4) で $k \overset{def}{\equiv} q$ と $k \overset{def}{\equiv} n$ として指数をそろえ、e と $\mathrm{comm}_{\mathbb{N}}$ を用いる）。
(6)（toZ∘fromZ）$\mathrm{toZ}(g^m) = [\langle m, \mathtt{0}\rangle]$ と $\mathrm{toZ}(h^n) = [\langle \mathtt{0}, n\rangle]$ を帰納法で示し、elimProp により $\mathrm{toZ}(\mathrm{fromZ}\, z) = z$ を示せ。
(7)（fromZ の準同型性）elimProp により $\mathrm{fromZ}(z +_{\mathbb{Z}} w) = \mathrm{fromZ}(z) \odot \mathrm{fromZ}(w)$ を示せ。また、$x \odot y = \mathtt{e}$ ならば $y = \mathtt{inv}\, x$ であることを示し、これと (4) から $\mathrm{fromZ}(\mathtt{neg}\, z) = \mathtt{inv}\,(\mathrm{fromZ}\, z)$ を導け。
(8)（fromZ ∘ toZ）Theorem 84（ii）と同様に、elimProp により $\mathrm{fromZ}(\mathrm{toZ}\, w) = w$ を示せ。$\odot$ と `inv` の場合に (7) を用いる。

History and Further Reading

高次帰納型は homotopy type theory の文脈で導入され（circle・suspension・truncation・商型などの例とともに HoTT book (The Univalent Foundations Program, 2013) の第 6 章で展開された）、その意味論は Lumsdaine and Shulman (2020) により与えられた。cubical type theory における高次帰納型の構成的な扱いが Coquand et al. (2018)（本章前半の転記元）であり、合成を hcomp と trans（本書の transp）に分解するその手法は、cartesian 側では Cavallo and Harper (2019) が対応する。

本章の整数型のように、商を高次帰納型として表現する試みには複数の表現が知られている。本章の $\mathbb{Z}$（Definition 57）は、同値関係の全体を path 構成子 `eq` とし set truncation を添える**商型スタイル**であり、古典的な差分の商による構成を最も直接に高次帰納型化したものである。successor と predecessor の同型を構成子として持つ別の表現を与え、truncation なしで set となることの証明を含めて詳述している。生成元を 1 本に絞って truncation を消す cancel 版は Section 5.1 で与えた（Definition 57 の脚注も参照）。

自由代数を高次帰納型として構成する話題（Section 4.8）は、自由群の構成を含めて HoTT book (The Univalent Foundations Program, 2013) の §6.11 に始まる。等式公理をもつ代数的理論一般の自由代数については、quotient inductive type としての定式化（Kaposi–Kovács–Altenkirch による POPL 2019 の仕事）と、set 切り詰めつき高次帰納型による普遍代数の展開（van der Weide–Geuvers）が知られている。

実装としては、Cohen et al. (2018) に付随する実験的実装 cubicaltt に始まり、現在では Agda の cubical モード (Vezzosi et al., 2019) が広く用いられている。本章の構成は、講義用リポジトリ `https://github.com/DaisukeBekki/AgdaTutorial-LatticeTheory` の第 IV 部（IntCore・Int・IntEffective・IntProperties の各モジュール。同じ eq + trunc 版の $\mathbb{Z}$、recSet ／ elimProp、正規形と効果性）に対応しており、機構部分は Cubical Agda で型検査済みである。また Section 5.1 の cancel 版 $\mathbb{Z}_c$ とその定理群（正規

形定理・集合性・ua による型値再帰を含む）は、同リポジトリの IntCancel モジュールで全主張が型検査済みである。とくに、cancel 版で「判断的に成り立つ」と主張した等値性（norm の 3 等式・toInt_c の cancel 節の定数性・正規形定理の充填・ua の端点）は、いずれも開いた変数のまま refl として型検査を通過することが確認された。

Bibliography

Altenkirch, T., C. McBride, and W. Swierstra. (2007) "Observational equality, now!", In Proceedings of *the 2007 Workshop on Programming Languages meets Program Verification (PLPV '07)*, A. Stump and H. Xi (eds.). pp.57–68.

Angiuli, C., G. Brunerie, T. Coquand, R. Harper, K.-B. Hou (Favonia), and D. R. Licata. (2021) "Syntax and models of Cartesian cubical type theory", *Mathematical Structures in Computer Science* **31**(4), pp.424–468.

Angiuli, C., R. Harper, and T. Wilson. (2017) "Computational higher-dimensional type theory", In Proceedings of *the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. pp.680–693, ACM.

Awodey, S. and M. A. Warren. (2009) "Homotopy theoretic models of identity types", *Mathematical Proceedings of the Cambridge Philosophical Society* **146**(1), pp.45–55.

Bekki, D. (2014) "Representing Anaphora with Dependent Types", In Proceedings of *Logical Aspects of Computational Linguistics (8th international conference, LACL2014, Toulouse, France), LNCS 8535*, N. Asher and S. V. Soloviev (eds.). pp.14–29, Springer, Heidelberg.

Bekki, D. and K. Mineshima. (2017) "Context-Passing and Underspecification in Dependent Type Semantics", In: S. Chatzikyriakidis and Z. Luo (eds.): *Modern Perspectives in Type-Theoretical Semantics*, Vol. 98 of *Studies in Linguistics and Philosophy*. Springer, pp.11–41.

Bezem, M., T. Coquand, and S. Huber. (2014) "A Model of Type Theory in Cubical Sets", In Proceedings of *19th International Conference on Types for Proofs and Programs (TYPES 2013)*, R. Matthes and A. Schubert (eds.), Vol. 26 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.107–128, Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

Cavallo, E. and R. Harper. (2019) "Higher Inductive Types in Cubical Computational Type Theory", *Proceedings of the ACM on Programming Languages* **3**(POPL), pp.1:1–1:27.

Cohen, C., T. Coquand, S. Huber, and A. Mörtberg. (2018) "Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom", In Proceedings of *21st International Conference on Types for Proofs and Programs (TYPES 2015)*, T. Uustalu (ed.), Vol. 69 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.5:1–5:34, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Preprint: arXiv:1611.02108.

Coquand, T., S. Huber, and A. Mörtberg. (2018) "On Higher Inductive Types in Cubical Type Theory", In Proceedings of *LICS '18: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.255–264.

Hofmann, M. (1995) "Extensional concepts in intensional type theory", Ph.D. thesis, University of Edinburgh.

Hofmann, M. and T. Streicher. (1998) "The groupoid interpretation of type theory",

In: G. Sambin and J. M. Smith (eds.): *Twenty-Five Years of Constructive Type Theory (Venice, 1995)*, Oxford Logic Guides, vol.36. New York, Oxford University Press.

Huber, S. (2019) "Canonicity for Cubical Type Theory", *Journal of Automated Reasoning* **63**(2), pp.173–210.

Kapulkin, K. and P. L. Lumsdaine. (2021) "The simplicial model of Univalent Foundations (after Voevodsky)", *Journal of the European Mathematical Society* **23**(6), pp.2071–2126.

Lumsdaine, P. L. and M. Shulman. (2020) "Semantics of higher inductive types", *Mathematical Proceedings of the Cambridge Philosophical Society* **169**(1), pp.159–208.

Martin-Löf, P. (1975) "An intuitionistic theory of types", In: H. E. Rose and J. Shepherdson (eds.): *Logic Colloquium '73*. Amsterdam, North-Holland, pp.73–118.

Martin-Löf, P. (1984) *Intuitionistic Type Theory*, Vol. 17. Naples, Italy: Bibliopolis. Sambin, Giovanni (ed.).

Mörtberg, A. (2021) "Cubical methods in homotopy type theory and univalent foundations", *Mathematical Structures in Computer Science* **31**(10), pp.1147–1184.

Nordström, B., K. Petersson, and J. Smith. (1990) *Programming in Martin-Löf's Type Theory*. Oxford University Press.

Nordström, B., K. Petersson, and J. M. Smith. (2000) "Martin-Löf's Type Theory", In: *Handbook of Logic in Computer Science, Vol 5*. Oxford University Press.

Orton, I. and A. M. Pitts. (2016) "Axioms for Modelling Cubical Type Theory in a Topos", In Proceedings of *25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, J.-M. Talbot and L. Regnier (eds.), Vol. 62 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.24:1–24:19, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Journal version: Logical Methods in Computer Science 14(4:23), 2018.

Sterling, J. and C. Angiuli. (2021) "Normalization for Cubical Type Theory", In Proceedings of *36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*. pp.1–15, IEEE.

Streicher, T. (1993) "Investigations into Intensional Type Theory", Ph.D. thesis, Habilitationsschrift, Ludwig-Maximilians-Universität.

The Univalent Foundations Program. (2013) *Homotopy Type Theory: Univalent Foundations of Mathematics*. `https://homotopytypetheory.org/book`.

Vezzosi, A., A. Mörtberg, and A. Abel. (2019) "Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types", *Proceedings of the ACM on Programming Languages* **3**, pp.87:1–87:29.